

CSCE 676 DATA MINING · MODULE 06 · LECTURE NOTES

Streams and Sketches

One pass, bounded memory, a stated error.

MODULE OUTCOMES

- Explain the streaming model and its constraints.
- Explain reservoir sampling and why it is uniform.
- Explain the counting sketches and their guarantees.
- Explain cardinality estimation.
- Verify a sketch's error bound empirically.

1 The model

ONE PASS, MEMORY SUBLINEAR IN THE STREAM LENGTH, AND AN ANSWER WITH A STATED ERROR

The stream is unbounded or simply too large to store, each item is seen exactly once, and the memory available is polylogarithmic in the stream length — **which rules out storing the data and therefore rules out exact answers to most questions** (§4's lower bounds).

So every streaming result is a trade between memory, accuracy, and failure probability — and **stating all three is what makes it a result rather than a heuristic**, which is Module 01 §2's discipline applied to this setting.

And this is the realistic model considerably more often than it appears: log processing, network flow monitoring, click streams, and sensor data all have this shape, **and so does any dataset larger than memory being read sequentially** — which is most of this course's data.

Which is Module 01 §2's cost model taken to its limit — **one pass is the minimum possible, and several of these algorithms achieve it** while answering questions that look like they should need the whole dataset.

2 Sampling

```
KEEP k ITEMS. For the i-th item ( $i > k$ ):  
    with probability  $k/i$ , replace a uniformly chosen  
    one of the k held items with it.  
  
THE CLAIM  
    after n items, every item has probability  $k/n$  of  
    being in the reservoir -- and this holds at every  
    n, without ever knowing n in advance.  
  
WHY (sketch, for  $k = 1$ )  
    item i is kept at step i with probability  $1/i$ , and  
    survives each later step j with probability  
     $(j-1)/j$ . The product telescopes:  
     $(1/i) * (i/(i+1)) * \dots * ((n-1)/n) = 1/n$   
  
WHICH IS THE WHOLE TRICK: the acceptance probability  
    falls exactly as fast as the survival probability  
    accumulates.  
  
AND FOR WEIGHTED OR SLIDING-WINDOW SAMPLING the  
    construction changes; do not assume this version  
    generalises.
```

The telescoping product is the proof, and it is worth writing out once — it explains why the k/i acceptance probability is exactly right rather than approximately right, and it makes the algorithm memorable rather than magical. The not-knowing-n property is what makes it useful in a stream: a sample drawn with a fixed probability requires knowing the length, and this does not.

3 Counting sketches

Sketch	What it answers	The guarantee
Bloom filter	Is x in the set?	No false negatives, and a tunable false positive rate — the asymmetry is the design.
Count-Min	How often did x occur?	Never underestimates; a bounded overestimate — see the callout.
Count-Sketch	The same question.	Unbiased, with a variance bound — the error can go either way, which suits different uses.
HyperLogLog	How many distinct items were there?	About 2% relative error in a few kilobytes, regardless of the cardinality — which is remarkable.
Misra-Gries	Which items are frequent?	Finds every item above a frequency threshold, plus some extras — so it is a candidate generator.

'Never underestimates' is Count-Min's key property, and it is worth dwelling on: the error is one-sided, which makes the sketch safe for threshold tests where a miss would be worse than a false alarm — if the estimate is below the threshold, the true count certainly is too. That one-sidedness is what makes it usable in practice for tasks like rate limiting and heavy-hitter detection.

COUNT-MIN WORKS BY TAKING THE MINIMUM OVER SEVERAL INDEPENDENT HASHES

Keep d rows of w counters each. On each arriving item, increment one counter per row, selected by that row's hash of the item. To query a count, take the minimum of the d counters the item hashes to.

Each individual counter is at least the item's true count, because collisions can only add to it and never subtract — so every one of the d estimates is an overestimate, and the minimum is simply the tightest of them. That is the whole argument for the one-sided guarantee.

And the error is bounded by the stream's total count divided by w , with a failure probability that falls exponentially in d — so the width controls the accuracy and the depth controls the confidence, which are two separate dials.

Which means the heavy hitters are estimated accurately and the tail is not — the error is *absolute* rather than *relative*, so a rare item's estimate can be wrong by a large multiple while the absolute error is within bound. This is §4's tail check and is the practical limitation of the structure.

4 Verifying, and what streaming cannot do

- Measure the actual error against the exact answer on a sample small enough to compute exactly — which is the whole verification, and it is cheap, and it is Module 01 §2's discipline in practice.

- **Compare the measured error to the stated bound, and investigate if the observed error is *better* than the bound as well** — which usually means the bound was loose for your data and occasionally means you measured the wrong thing.
- **Check the hash independence**, because **the guarantees assume pairwise or k-wise independent hash functions** and a fast non-cryptographic hash chosen for speed may not deliver it — which shows up as correlated collisions rather than as an obvious failure (Module 02 §4's same warning).
- **Check the tail specifically** — the error on rare items is exactly where Count-Min's absolute bound hurts, and **an aggregate error figure is dominated by the frequent items, which are the ones the sketch gets right. This is the step that matters.**
- **And measure the memory you actually used**, since **the space was the entire point of the exercise** — a sketch using more memory than the exact structure would have is a result worth discovering in testing.

AND WHAT STREAMING CANNOT DO

Exact distinct counts, exact medians, and exact quantiles all require memory linear in the number of distinct items — and these are proved lower bounds rather than missing algorithms, which is worth knowing so you stop looking.

Anything that needs a second look at an earlier item — so joins, sorting, and any iterative algorithm are outside the pure streaming model, and the practical systems of Module 10 exist partly to relax the one-pass constraint.

And sliding windows are substantially harder than prefixes, because expiry requires knowing what to forget, which none of §3's sketches can do — they accumulate monotonically. Windowed variants exist and cost more.

So the honest framing is: streaming gives you approximate answers to a specific and limited set of questions, in exchange for a single pass — and knowing which questions are on that list is the content of this module, more than the individual algorithms are.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Mining of Massive Datasets, chapter 4 (free)	The whole module — the streaming model, the sketches, and the sampling, with the analyses.
Cormode & Muthukrishnan — An improved data stream summary (free)	§3's Count-Min in the original , with the bound derived — short and readable.
Flajolet et al. — HyperLogLog (free)	§3's fourth row — and the analysis is a genuinely elegant piece of work.

Resource	What it covers for this module
Cormode & Yi — Small Summaries for Big Data	§3 and §4 comprehensively — including the lower bounds and the windowed variants. Library copy.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement reservoir sampling** and verify uniformity empirically over many runs.
- 2 **Write out the telescoping product** for $k = 1$.
- 3 **Implement a Bloom filter** and measure the false positive rate against its prediction.
- 4 **Implement Count-Min** and verify it never underestimates.
- 5 **Measure its error for frequent and rare items separately.**
- 6 **Vary the width and the depth** and confirm which controls accuracy and which confidence.
- 7 **Implement HyperLogLog** and measure the relative error at three cardinalities.
- 8 **Replace the hash with a deliberately poor one** and report what happens to the bounds.
- 9 **Measure the memory used** against the exact structure.
- 10 **Try to compute an exact median in one pass** and explain why you cannot.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the streaming model's three constraints.
- 2 Why is every streaming result a three-way trade?
- 3 Give reservoir sampling and the claim it satisfies.
- 4 Explain why the telescoping product gives $1/n$.
- 5 Name five sketches and what each answers.
- 6 What is Count-Min's key property, and why does it matter?
- 7 Explain the mechanism and what width and depth each control.
- 8 Why is the tail inaccurate?
- 9 Give five verification steps and the one that matters most.
- 10 Name three things streaming cannot do.